

pw-26th nov-5

—auto suggestions.

```
//use for each loop.
test("handle auto complete suggestions like google search", async({page})=>{

  await page.goto("https://www.google.com/");

  await page.pause();

  //goto google search and type and slowly type hence delay.
  await page.locator("//textarea[@id='APjFqb']").type("Mukesh Otwani", {delay:200})

  //get all the suggestions shown once we start searching
  let elementsonsearch=await page.locator("//div[@role='option']")

  //count how many elements preent in dom
  let countofelements= await elementsonsearch.count();
  console.log(countofelements)

  //use alltextcontents to capture each value
  const values=await elementsonsearch.allTextContents();

  //use for each to print the values
  //first is v1 which will hold each value
  //second is index number it captures during run
  //third is array on which to run and manipulate
  values.forEach(function(v1,i1,a1){
    console.log(v1);

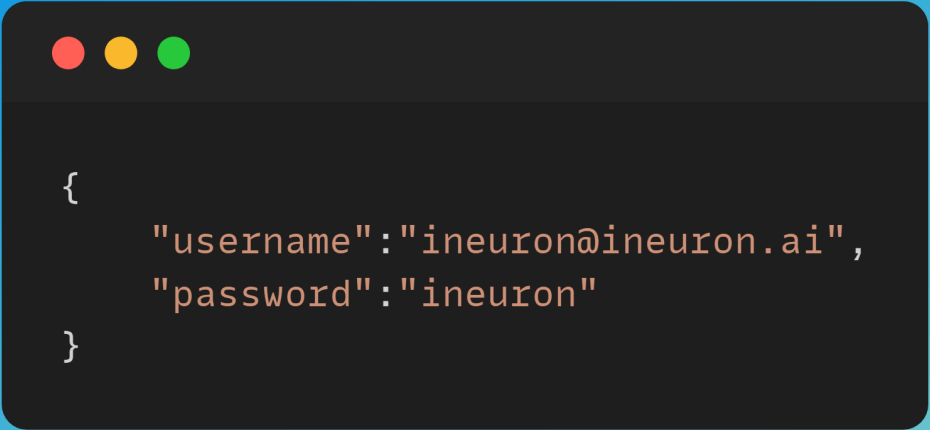
    if(v1.includes("api")){

      //nth method to click on the element
      //here we pass index like array
      elementsonsearch.nth(i1).click();
    }
  })

  //to search for partial strings we can pass like this.
  await expect(page).toHaveTitle(/.*api/)

})
```

test data in json format and access in our code.



```
{  
  "username": "ineuron@ineuron.ai",  
  "password": "ineuron"  
}
```

codesnap.dev

```

import {test, expect} from '@playwright/test'

//get data from json file.
let v1=require("../tests/testdata/mytestdata.json")
//convert to json string
let v2= JSON.stringify(v1);
//now to parse the json we need to use parse so we can use json.parse
let data=JSON.parse(v2)

test("login logout with the test data", async({page})⇒{

    //go to page
    await page.goto("https://ineuron-courses.vercel.app/");

    //click login
    await page.getByText("Log in ").click()

    //check url
    await expect(page).toHaveURL("https://ineuron-courses.vercel.app/login");

    //type user name , password
    await page.getByPlaceholder("Email").type(data.username)
    await page.getByPlaceholder("Password").type(data.password)

    //click sign in button and check url once signed in
    await page.getByRole("button",{name:'Sign in' }).click()
    await expect(page).toHaveURL("https://ineuron-courses.vercel.app/")

    //click on sign out button and check url once signed out
    await page.getByRole("button",{name:'Sign out' }).click()
    await expect(page).toHaveURL("https://ineuron-courses.vercel.app/login")
})

```

codesnap.dev

—multiple test data example, parsing is same initially.

```

//store multiple test data in form of json array
const v1=[
  {
    "username":"ineuron@ineuron.ai",
    "password":"ineuron"
  },
  {
    "username":"sample9@gmail.com",
    "password":"sample9@gmail.com"
  },
  {
    "username":"testuser101@gmail.com",
    "password":"testuser101@gmail.com"
  }
]

//same test only with multiple data.
test("check with multiple test data", async({page})=>{
  for (const data of v1) {
    await page.goto("https://ineuron-courses.vercel.app/")

    await page.getByText("Log in").click()
    await expect(page).toHaveURL('https://ineuron-courses.vercel.app/login')

    await page.getByPlaceholder("Email").type(data.username)
    await page.getByPlaceholder("Password").type(data.password)

    await page.getByRole("button",{name:'Sign in' }).click()
    await expect(page).toHaveURL("https://ineuron-courses.vercel.app/")

    await page.getByRole("button",{name:'Sign out' }).click()
    await expect(page).toHaveURL("https://ineuron-courses.vercel.app/login")
  }
})

```

codesnap.dev

—
this one we can use for concatenation and it prints id.

test("multiple data with id " + [data.id](#), async({page})=>{

—concat method works, but we need to cast to string.

```
//same test only with multiple data but id field is added to identify every test case uniquely.
for (const data of v3) {

    test("multiple data with id ".concat(String(data.id)), async({page})=>{
```

codesnap.dev

—if we write like this, then we cant identify individual cases in report though test case passes.
so first for loop then run three times with different test names.

```
//same test only with multiple data.
test("check with multiple test data", async({page})=>{
    for (const data of v3) {
```

codesnap.dev

```
//same test only with multiple data.
//compile error when for loop is outside and we do data driven.
//what happens, it runs same test multiple times with same name.
//so thats why we need concat with some unique primary key for every test.
for (const data of v3) {
test("check with multiple test data and for loop outside", async({page})=>{
```

codesnap.dev

Change worker in pw config.

```

25     },
26     /* Run tests in files in parallel */
27     fullyParallel: true,
28     /* Fail the build on CI if you accidentally
29     forbidOnly: !!process.env.CI,
30     /* Retry on CI only */
31     retries: process.env.CI ? 2 : 0,
32     /* Opt out of parallel tests on CI. */
33     //workers: process.env.CI ? 1 : undefined,
34     workers: 3,
35     /* Reporter to use. See https://playwright.dev/docs/test-reports
36     reporter: 'html'

```

how workers work-

In workers the files run parallelly not the tests. If one file has four test, they will run sequentially, but the files themselves run in parallel. Recommended number of workers between 3-5. When running parallelly on different browsers, first all workers will run on one browser then all workers on second browser and so on.

—Run specific file-

```

✓ TERMINAL
mukeshotwani@Mukeshs-MacBook-Air-2 Playwright % npx playwright test ./tests/loginlogout.spec.
js

```

—

Run on specific project- Project is browsers in pw.

```

mukeshotwani@Mukeshs-MacBook-Air-2 Playwright % npx playwright test ./tests/loginlogout.spec.
js --project=chromium
Running 1 test using 1 worker

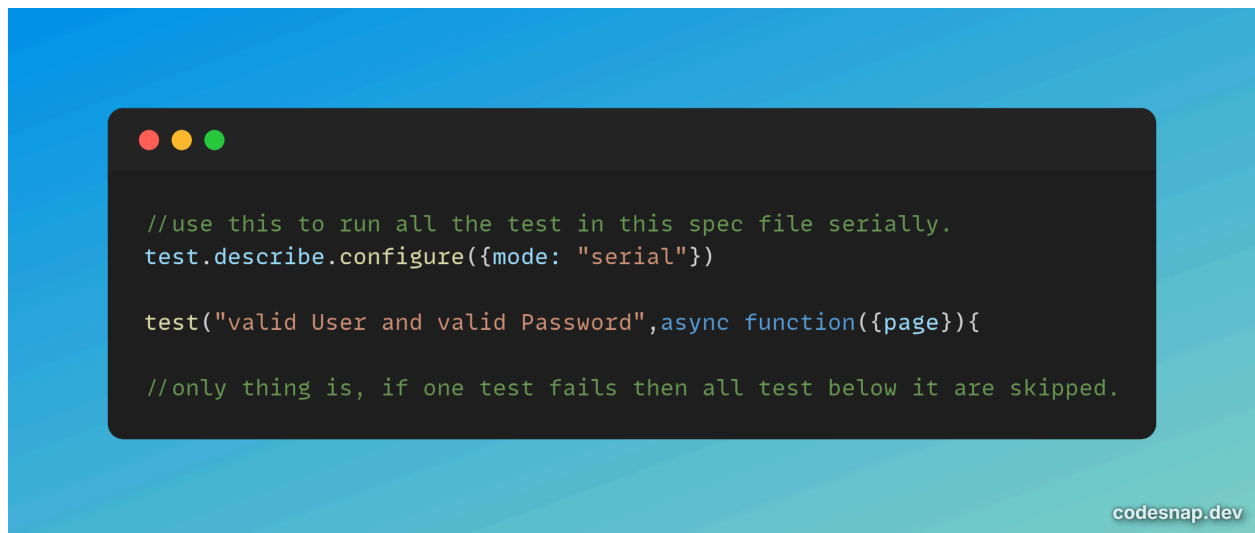
```

—Run all tests/file-

```
3  
4   Run all test/file  
5   npx playwright test  
6
```

—

only line to add to run test inside spec file serially.

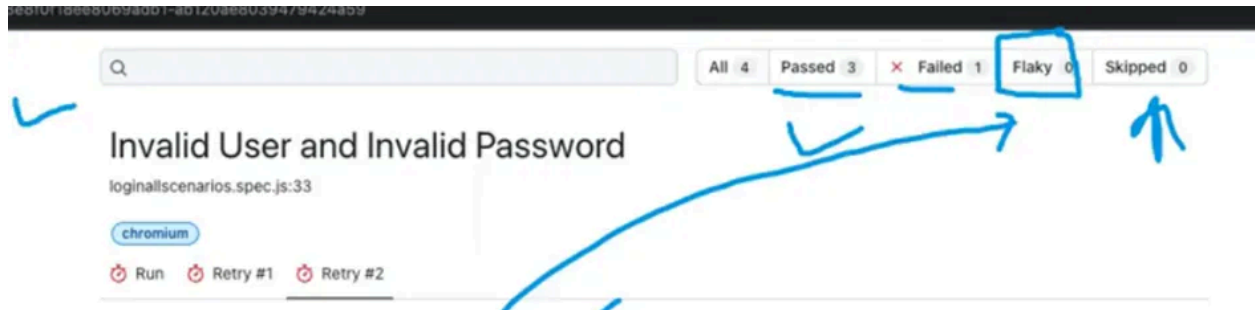


—retry analyser for failed tests.

Pw config- change retries number and run on local.

```
31   //retries: pr  
32  
33   retries:2, }
```

—flaky test in report- If second retry passed then it will mark the test as flaky.



—Multiple config allowed in pw as per our needs-

we can then pass this file in our require statement.



we can also tell to use the config1 in our command line.

```
mukeshotwani@Mukeshs-MacBook-Air-2 Playwright % npx playwright test ./tests/loginallscenarios.spec.js --config=playwright1.config.js
```

if you dont specify the config here in cmd, then default config is picked.